Hierarchical Rule-Base Reduction-Based ANFIS With Online Optimization Through DDPG

Marius F. R. Juston, Samuel R. Dekhterman, William R. Norris, *Member, IEEE*, Dustin Nottage, and Ahmet Soylemezoglu

***Abstract*—This article presents a comprehensive approach to designing and optimizing a hierarchical rule-base reduction-based adaptive-network-based fuzzy inference system (ANFIS) for symmetric linguistic variables. Specifically, the linguistic connected membership functions that underlie the ANFIS are defined, focusing on symmetrical inputs/outputs and jointly optimized trapezoid membership functions to reduce the number of training parameters. Further optimizations for the ANFIS were derived based on design assumptions, including training the membership functions on closed or single-sided domains. The optimal output membership weights based on mean square error optimization were also symbolically obtained. The online training of the ANFIS's input/output membership functions was performed using the deep deterministic policy gradient (DDPG) algorithm. A simulated skid-steered vehicle was used to validate the approach and performed waypoint-to-waypoint path following. Experimental results using the Clearpath Jackal demonstrated that the ANFIS model converged quickly, typically within 6 to 10 episodes of training, from an initial mean absolute error (MAE) and root mean squared error (RMSE) of 0.88 and 1.02 m, respectively, to a final MAE and RMSE of 0.087 and 0.10 m. The results highlight the effectiveness of the ANFIS approach for vehicular robotics applications and suggest promising avenues for future research and development.**

***Index Terms*—Adaptive-network-based fuzzy inference system (ANFIS), deep deterministic policy gradient (DDGP), fuzzy logic, hierarchical rule-base reduction (HRBR), robot operating system (ROS), reinforcement learning, skid-steer vehicle.**

I. Introduction

With the recent rise to prominence of reinforcement learning, a variety of applications related to control have been investigated [1], [2], [3], [4], [5], [6]. In these applications, reinforcement learning and control in robotics are usually independent [7], [8], [9]. Most systems use reinforcement learning and deep neural networks to have robots dynamically follow set goals given a full suite of sensor inputs [3], [10]. However, developing and training the networks can take a great deal of time. Depending on the task's complexity, such as modeling human motion, either methodology can take days or weeks of GPU time to train a model to perform accurately [11]. Furthermore, due to the closed-box nature of neural networks, manually adjusting the parameter weights to obtain a marginally different set of desired output characteristics is difficult [12]. Despite these issues, reinforcement learning is an extremely powerful and versatile tool. Such versatility results from systems not needing underlying information on their operating environment [10]. In robotics, this is seen from the use of underlying convolution neural networks for understanding camera feeds [13], [14], [15], [16], [17] to natural language processing for human-to-robot interactions [18], [19], [20], [21], [22], [23], [24], [25], [26], [27]. Conversely, classical control system designs are far less versatile, require explicit model selection/design, and often some hand tuning [28].

However, one strength of classical model-based control methodologies is the minute understanding of the system's characteristics. Such knowledge enables engineers to set bounds on convergence and system errors, which allows the controller to reach a steady state within guaranteed time bounds, have a certain level of overshoot, and be accurate to within a certain tolerance [29], [30]. There are also model-free classical control methodologies, such as fuzzy logic control, PID loops, model-free adaptive control, and more [31], [32]. The convergence of such models can also be ensured through Lyapunov functions, and passivity [29].

The classical model-free controller provides an advantage over neural-network-based reinforcement learning, where proofs of convergence to particular equilibrium points are more challenging. To join these concepts, an adaptive-network-based fuzzy inference system (ANFIS) was developed to establish a differentiable fuzzy logic process [33], [34]. The ANFIS has the advantage of being able to employ traditional classical model-free mechanisms in a human-interpretable fashion, which enables expert operators to modify the underlying rulesets and membership functions of the model with little difficulty; the additional integration of the deep deterministic policy gradient (DDPG) algorithm is used to train the

Received 13 October 2023; revised 7 May 2024, 11 June 2024, and 9 July 2024; accepted 21 August 2024. Date of publication 23 August 2024; date of current version 1 November 2024. This work was supported in part by Construction Engineering Research Laboratory under Grant W9132T23C0013. Recommended by Associate Editor J. Fernandez. *(Corresponding author: Marius F. R. Juston.)*

Marius F. R. Juston is with the Department of Industrial and Enterprise Systems Engineering, University of Illinois Urbana-Champaign, Champaign, IL 61801 USA (e-mail: mjuston2@illinois.edu).

Samuel R. Dekhterman is with the Department of Industrial and Enterprise Systems Engineering, University of Illinois Urbana-Champaign, Champaign, IL 61801 USA (e-mail: srd2@illinois.edu).

William R. Norris is with the Department of Industrial and Enterprise Systems Engineering, University of Illinois Urbana-Champaign, Champaign, IL 61801 USA (e-mail: wrnorris@illinois.edu).

Dustin Nottage and Ahmet Soylemezoglu are with the Construction Engineering Research Laboratory, U.S. Army Corps of Engineers Engineering Research and Development Center, Champaign, IL 61822 USA (e-mail: dustin.s.nottage@erdc.dren.mil; ahmet.soylemezoglu@erdc.dren.mil).

Digital Object Identifier 10.1109/TFUZZ.2024.3449147

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

ANFIS's parameter set adaptively. This article focuses on applying the ANFIS in robotics motion control. The approach is to implement constraints on the ANFIS's structure to maintain theoretical guarantees during the network's unconstrained gradient-based training. The main constraint implemented is a symmetry constraint, where the ANFIS should maintain the property $y(x) = -y(-x)$, as well as an additional constraint of positive definiteness. In robotic applications, control symmetry can be applied as most motion models perform symmetrically based on their control inputs [35], [36]. As such, when driving a car, turning left turns left, and in the same idea, turning right the same amount will turn the car right the same amount. This symmetrical constraint reduces the optimization search space, as it would only be required to train one side of the domain. In turn, the total number of optimization parameters also decreases, improving converge time.

This article introduces the following innovations.

1) The development of constrained and implicitly symmetrically bounded linguistic variable membership functions.
2) An optimized definition of the Mamdani rule-base fuzzy control of the ANFIS.
3) The parameter-set training of the ANFIS in an online fashion through (DDPG) reinforcement learning.

The rest of this article is organized as follows. This article will start with the definitions and terminology of rule-based fuzzy control and the ANFIS in Sections II and III, respectively. Section IV then transitions to the thought processes and implementation of the symmetrically and jointly bounded trapezoid membership functions. In addition, the trapezoid membership functions are then placed in the ANFIS system, whose mathematical definition is reduced to a series of linear operations. Sections V and VI discuss offline and online training with online training incorporating DDPG. Results follow for the online case in Section VII. Finally, Section VIII concludes this article.

II. Fuzzy Inference System

Fuzzy inference systems employ IF–THEN rules and use underlying human knowledge to exert effective control, expressed in the form of the following:

$$\text{IF } A \text{ AND/OR } B \text{ THEN } O.$$

The linguistic variables A and B [37] are labels inside fuzzy sets attributed to a specific membership function [38]. For a more human-facing example

$$\text{IF rain is heavy AND speed limit is high}$$
$$\text{THEN velocity} = \text{low}.$$

A hierarchical rule-base reduction (HRBR) [39], [40], [41], [42] was introduced, developed, and used to systematically define the minimum rule-set for a given problem using human knowledge to categorize and reduce computational complexity. Fuzzy inference systems, usually consisting of four steps, are devised to model and evaluate fuzzy rule sets. This rule-base reduction is different from the works originally introduced by Sugeno and later extended in [43], [44], [45], [46], and [47], where a structure of fuzzy systems is used to hierarchize the architecture. The hierarchical approach discussed in this article focuses instead on categorizing the rules themselves.

1) From a crisp input, compute the membership level (degree of matching) in each of the input linguistic variable's linguistic values, often called the fuzzification step.
2) Compute the firing strength of each rule through the combination of the membership values for the inputs. The combination operator was implemented through a T-Norm operation. A T-Norm represents models the fuzzy connective AND [48], with the T-Conorm representing the fuzzy connective OR [49]. Other functions can also model this connective, such as overlap functions [50]. This step represents the "rain is heavy AND speedlimit is high" combination in the rule-base.
3) The output of each rule is then associated with the corresponding consequent membership value. This represents the "IF... THEN set velocity to low" operation in the rule-base.
4) Finally, the outputs of the rules are aggregated to produce a crisp output. This step is called defuzzification.

III. Adaptive-Network-Based Fuzzy Inference System

A. Model Overview

The ANFIS is a mathematical model structured similarly to a fully connected neural network. While structured to perform rule-based fuzzy control, the model's design enables it to tune its variables through backpropagation dynamically [33]. The neural network approach was represented through five sequential layers, each of which was associated with a step in the rule-based fuzzy control inference system. Layers 2 and 3 represented step 2 of the fuzzy inference systems.

1) Layer 1: The calculation of the degree of matching (often called the fuzzification) of the crisp input $\bar{x} \in \mathbb{R}$ was accomplished by using trapezoidal membership functions, $\mu_{k,p}$, with each function being 0 for all input values not included in its trapezoidal section. For the membership functions, k specified the associated linguistic variable and p the given linguistic value

$$\mu_{k,p} = \text{trapezoid}_{k,p}(x). \tag{1}$$

2) Layer 2: This represents the weights of the outputs of the rules. For a given rule, i, a combination of the chosen input membership functions, $\mu_{k_1,p_1}, \mu_{k_2,p_2}$ determines the output membership's function firing rate, μ_i. The product T-Norm was chosen to combine the input membership functions and facilitated the gradient computation [51]

$$\mu_i = \mu_{k_1,p_1} \times \mu_{k_2,p_2}. \tag{2}$$

3) Layer 3: It is the normalization layer, where the output membership function firing rates are normalized

$$n_i = \frac{\mu_i}{\sum_{i=1}^{r} \mu_i}. \tag{3}$$

4) Layer 4: Is what represents the consequent portion of the network. Using triangle output membership functions and the center-of-maximum defuzzification implementation, the output for each rule was simply a scalar product of the peak of its

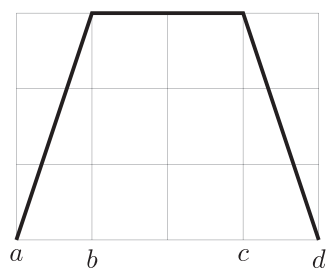

Fig. 1. Trapezoid.

associated output membership function and normalized firing rate.

$$z_i = \text{triangle}_{O_i} \times n_i. \tag{4}$$

5) Layer 5: Is where these weighted rule outputs are combined into a crisp output

$$y = \sum_{i=1}^{r} z_i. \tag{5}$$

B. Input Membership Function

This section explains additional constraints on the input trapezoid membership functions that made the system characteristics and training unique.

1) Classical Trapezoid: This section uses a trapezoid example to provide background for later implementation and constraint derivations.

As seen in Fig. 1, a trapezoid was defined using four parameters $a, b, c, d \in \mathbb{R}$ constrained as $a \leq b \leq c \leq d$

$$\text{trapezoid}(x) = \mu(x) = \max\left(\min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right), 0\right). \tag{6}$$

2) Connected Membership Functions: In control systems, "bang–bang" refers to rapidly switching between two extreme values in response to noisy or fluctuating input signals. This behavior can cause high-frequency outputs that can harm the system, leading to malfunctions due to oscillations and generating less smooth results. Trapezoidal membership functions are sometimes used instead of other membership functions in rule-based fuzzy control systems to mitigate this issue. Trapezoidal functions have flat regions that provide a margin of acceptable error in the input, especially around the zero error region, which can reduce the effect of noisy inputs and improve overall system stability. Using trapezoids also allows for some of the desirable traits of a Gaussian function to be captured computationally efficiently, as shown in [52]. This can lead to a better performance and more accurate results while minimizing the potential adverse effects of bang–bang behavior on the system. A novel idea was the implementation of connected membership functions. A constraint imposed on the system was that the sum of all the membership functions was equal to one, denoted as a Ruspini-partition [53], [54], at every point in each linguistic

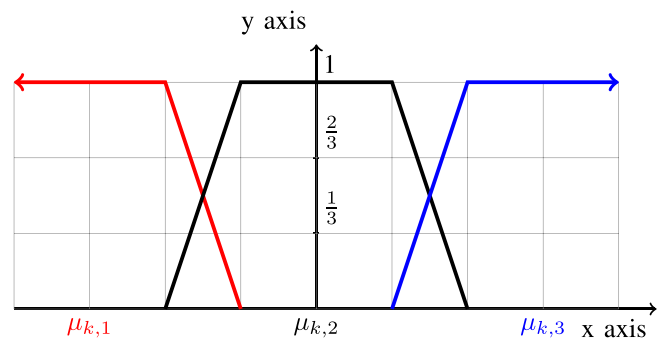

Fig. 2. Connected membership function example.

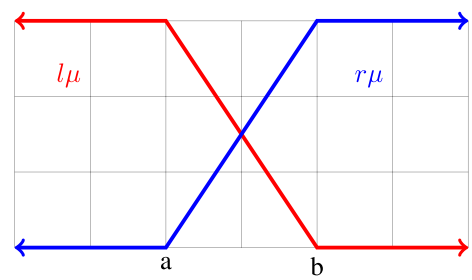

Fig. 3. Single-sided membership function definitions.

variable domain (see Fig. 2). As such

$$\sum_{i=1}^{p_k} \mu_{k,p_i}(x) = 1 \quad \forall k, x. \tag{7}$$

A demonstration of connected membership functions over the domain of a linguistic variable is further depicted in Fig. 2.

Through this constraint, the membership variables a, b, c, d for each trapezoid depended on each other. The boundary trapezoids were defined as single-sided functions (see Fig. 3)

$$l\mu(x) = \begin{cases} 1, & \text{if } x < c \\ \frac{x-d}{c-d}, & \text{if } c \leq x < d \\ 0, & \text{if } x \geq d \end{cases} \tag{8}$$

$$r\mu(x) = \begin{cases} 0, & \text{if } x < a \\ \frac{x-a}{b-a}, & \text{if } a \leq x < b \\ 1, & \text{if } x \geq b. \end{cases} \tag{9}$$

The following example illustrates the membership functions associated with a linguistic variable possessing five linguistic values (see Fig. 4).

The trapezoid functions parameters were, thus, defined relative to each other $i = 1, 2, \ldots, p_k \quad \forall k$

$$c_i = a_{i+1}, \quad d_i = b_{i+1}. \tag{10}$$

The variable p_k was defined as the number of membership functions for a specific linguistic variable. Accordingly, there were k linguistic variables. This constraint (7) helped to reduce the parameter set size from $4(p-1)$ to $2(p-1)$. The parameter set size was further reduced by assuming that the system was

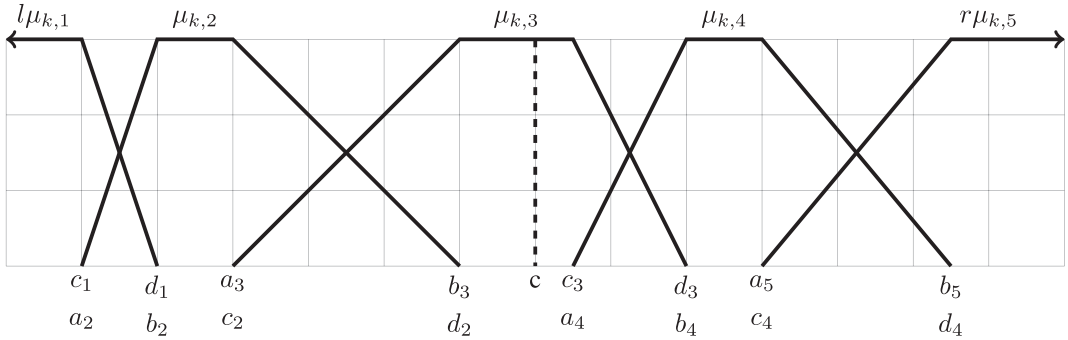

Fig. 4. Connected membership functions variables.

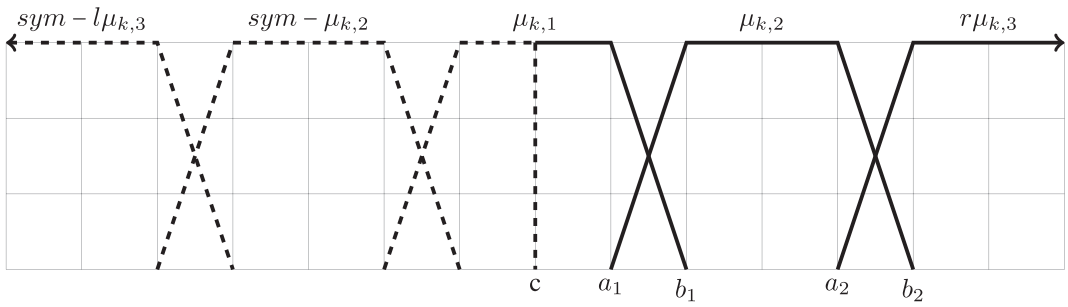

Fig. 5. Reduced symmetric connected membership functions.

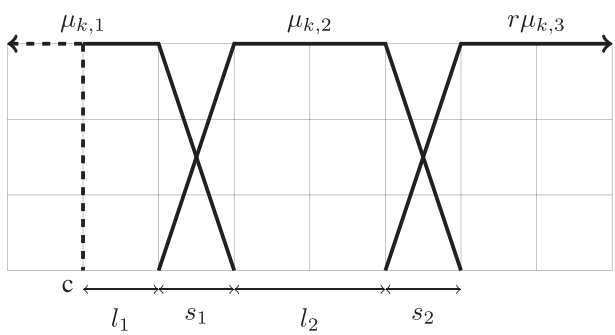

Fig. 6. Reparametrized symmetric connected membership functions.

symmetric along axis c and then only defining the membership functions on one side of the axis (see Fig. 5).

The symmetry constraint further reduced the parameter set to a size p. For all these scenarios, it was assumed that $p \geq 2$; otherwise, the membership function would be defined as $\mu(x) = 1, \forall x$, which is a trivial function. This indicates an unnecessary linguistic variable.

The parameters a_n, b_n, and c were defined as coordinates on the x-axis; however, to further introduce the constraint $\mathrm{c} \leq a_1 \leq b_1 \leq \ldots \leq a_n \leq b_n$ into the system, a redefinition of the variables was required. The variables were instead redefined as widths between the key points (see Fig. 6). These were represented by l_n and s_n, the width of the top and sloped regions of the trapezoids, respectively. This parameter redefinition allowed the system to occupy the same size p parameter set.

Through the redefinition of the parameter set, the $\mathrm{c} \leq a_1 \leq b_1 \leq \ldots \leq a_n \leq b_n$ constraint became

$$\begin{aligned} &\mathrm{c} \leq a_1 \leq b_1 \leq \ldots \leq a_n \leq b_n \\ &\mathrm{c} \leq \mathrm{c} + l_1 \leq \mathrm{c} + l_1 + s_1 \leq \ldots \leq \mathrm{c} + \sum l_i + \sum s_i. \end{aligned} \tag{11}$$

The constraint followed a nonnegative restriction and a cumulative sum relationship. Thus, constraining the trapezoid shape became computationally efficient.

3) Trapezoid Computation: The parameter set was defined as $\{w, \mathrm{c}\}$

$$w = \begin{cases} \begin{bmatrix} l_1 & s_1 & \ldots & l_n & s_n \end{bmatrix}^T, & \text{if } p_k \text{ is odd} \\ \begin{bmatrix} s_1 & l_2 & \ldots & l_n & s_n \end{bmatrix}^T, & \text{if } p_k \text{ is even.} \end{cases} \tag{12}$$

With the size of $\dim(w) = p - 1$. To enforce nonnegativity, the system was subjected to an element-wise squaring of w, (13), by using a Hadamard power

$$B = A^{\circ 2} \Rightarrow B_{ij} = A_{ij}^2.$$

The cumulative sum of the elements in w_s was used to compute the new set of nonnegative key points representing trapezoid functions

$$\begin{aligned} w_s &= w^{\circ 2} \\ t_c &= \begin{bmatrix} t_{c,1} & \ldots & t_{c,(p-1)} \end{bmatrix}^T \end{aligned} \tag{13}$$

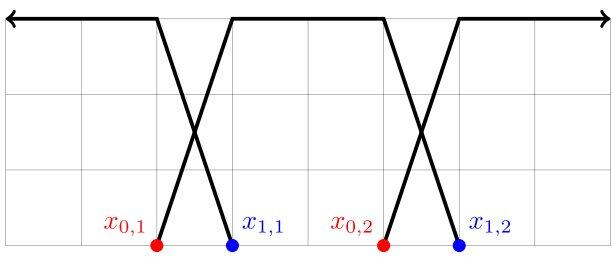

Fig. 7. x_0, x_1 coordinate set demonstration.

$$t_{ci} = \sum_{j=1}^{i} (w_s)_j \text{ for } i \in \{1, 2, \ldots, p-1\}. \tag{14}$$

The cumulative sum operation was represented as a lower triangular matrix C such that $C_{i,j} = 1$ if $j \leq i$, $C \in \mathbb{R}^{p-1 \times p-1}$

$$t_c = Cw_s. \tag{15}$$

To define the symmetrical output, the reflection operator matrix R was calculated such that $R \in \mathbb{R}^{2(p-1)\times(p-1)}$, $i, j \in \{1, 2, \ldots, p-1\}$

$$R_{i,j} = \begin{cases} -1, & \text{if } j = p - i \\ 1, & \text{if } j = (p-1) + i \\ 0, & \text{else} \end{cases} \tag{16}$$

$$R = \begin{bmatrix} & & -1 & 1 & & \\ & -1 & & & 1 & \\ -1 & & & & & 1 \end{bmatrix}^T$$

$$r = \begin{bmatrix} -t_{c,p-1} & \ldots & -t_{c,1} & t_{c,1} & \ldots & t_{c,p-1} \end{bmatrix}^T$$

$$r = Rt_c. \tag{17}$$

The vector r was reduced to $r = RCw_s = Mw_s$, where $M \in \mathbb{R}^{2(p-1)\times p-1}$. Once the reflected vector had been calculated, the vector was shifted to be symmetric around c

$$r_{\mathrm{c}} = \mathrm{c} + r. \tag{18}$$

Following the definition of the vector r_{c}, the vector was divided into two subvectors. These subvectors represented coordinates associated with the sloped regions of the trapezoids, with x_0 and x_1 associated with coordinates for the left and right sides of the sloped regions, respectively, $i \in \{1, \ldots, p-1\}$, $X_0, X_1 \in \mathbb{R}^{(p-1)\times 2(p-1)}$

$$x_{0,i} = r_{\mathrm{c},k}, \qquad k = 2i - 1 \tag{19}$$

$$x_{1,i} = r_{\mathrm{c},k}, \qquad k = 2i \tag{20}$$

$$\text{or } x_0 = X_0 r_{\mathrm{c}}, \quad x_1 = X_1 r_{\mathrm{c}}$$

$$X_{0_{i,k}} = 1, \qquad k = 2i - 1$$

$$X_{1_{i,k}} = 1, \qquad k = 2i \tag{21}$$

where $X_{0_{i,k}}, X_{1_{i,k}} = 0$ otherwise. A graphical representation of the x_0, x_1 coordinate subset is demonstrated in Fig. 7.

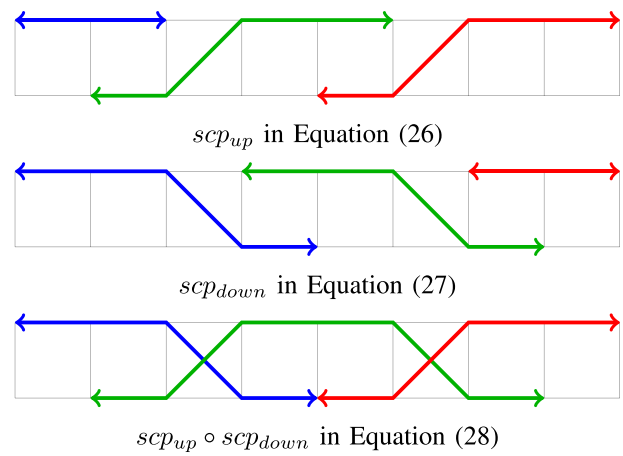

Fig. 8. Trapezoid creation in domain $x \in (-\infty, \infty)$.

The Hadamard division, $\oslash$, (element-wise vector division) was annotated as

$$C = A \oslash B \Rightarrow C_{ij} = \frac{A_{ij}}{B_{ij}}.$$

From the vector subsets x_0 and x_1, s_{up} defined the sloped regions, such that the scalar x was broadcasted to become a vector $x \in \mathbb{R}^{p-1\times 1}$

$$s_{\mathrm{up}}(x) = (x - x_0) \oslash (x_1 - x_0) \tag{22}$$

$$= (x - X_0 r_c) \oslash (X_1 r_c - X_0 r_c)$$

$$= (x - X_0 r_c) \oslash ((X_1 - X_0)\, r_c)\,. \tag{23}$$

Note, if r_c was zero, a division by zero error would arise, so an ϵ was added to the r_c component, $r_c \to r_c + \epsilon$. Following the creation of the linear region, clamping was used to define the regions of zero slope. As a result, $\mathrm{sc}_{\mathrm{up}}(x)$ represented the increasing sides of the trapezoids, and $\mathrm{sc}_{\mathrm{down}}(x)$ represented the decreasing sides of the trapezoids. These sides were bounded such that, $\mathrm{sc}_{\mathrm{up}}(x), \mathrm{sc}_{\mathrm{down}}(x) \in [0, 1]$ with dimensions $\mathbb{R}^{p-1\times 1}$ and

$$\mathrm{sc}_{\mathrm{up}}(x) = \max\{\vec{0}, \min\{\vec{1}, s_{\mathrm{up}}(x)\}\} \tag{24}$$

$$\mathrm{sc}_{\mathrm{down}}(x) = \vec{1} - \mathrm{sc}_{\mathrm{up}}(x). \tag{25}$$

Finally, the vectors, $\mathrm{scp}_{\mathrm{up}}(x), \mathrm{scp}_{\mathrm{down}}(x) \in \mathbb{R}^{p\times 1}$, were padded and combined using Hadmard product ($C = A \circ B \to C_{i,j} = A_{i,j} B_{i,j}$) to produce the final trapezoid outputs (illustrated in Fig. 8)

$$\mathrm{scp}_{\mathrm{up}}(x) = \begin{bmatrix} 1 & sc_{\mathrm{up}}(x) \end{bmatrix} \tag{26}$$

$$\mathrm{scp}_{\mathrm{down}}(x) = \begin{bmatrix} sc_{\mathrm{down}}(x) & 1 \end{bmatrix} \tag{27}$$

$$\mathrm{trapezoid}(x) = \mathrm{scp}_{\mathrm{up}}(x) \circ \mathrm{scp}_{\mathrm{down}}(x). \tag{28}$$

4) Single Sided Constraint: This membership function assumed that the domain of the variable was restricted to a single

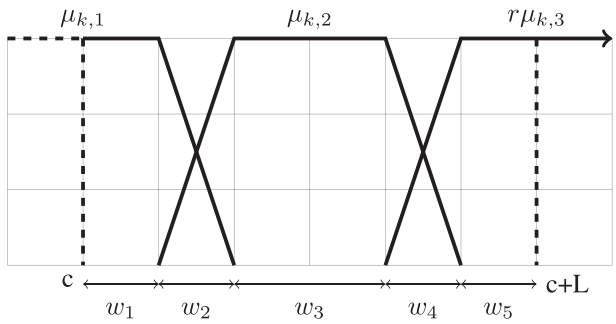

Fig. 9. Closed bound membership function.

direction, i.e., $[c, \infty)$. To satisfy this constraint, the function was restricted from using the reflection operator in (17).

5) Closed Domain Constraint: The previous implementation of the linguistic variable's connected membership functions assumed that the domain for the variable was $(-\infty, \infty)$ in $\mathbb{R}$. These variables could represent velocity or signed perpendicular distance to a line; however, some problems are bounded to $[a, b]$. These variables could be the robot's headings, which are bounded to $[-\pi, \pi]$. This section constrains the linguistic variable's membership functions within the symmetrical domain of $[a, b] = [c - L, c + L]$. The primary constraint on the system involved restricting the cumulative sum of the widths not to exceed distance L. A normalization step was performed on the squared width vector w_s, $i \in \{1, \ldots, p_k\}$ from (13)

$$w_{s,i} = \frac{w_i^2}{\sum_{i=1}^{p} w_i^2} L. \tag{29}$$

This constraint introduced an additional width variable w_{p_k}, as seen in Fig. 9 with w_5. However, following the aforementioned steps, only the $i \in \{1, \ldots, p-1\}$ weights were needed to compute the trapezoid functions. The contribution w_{p_k} provided was to act as a normalization constant to $\sum_{i=1}^{p} w_i^2$ and is also trained when computed in the gradients.

IV. ANFIS Model Optimization

A. Overview

Recall that the ANFIS was defined as comprising five different layers defined in (1)–(5). This section revolves around reducing the system of equations into a more compact model. The compact model followed a similar structure to the Takagi–Sugeno fuzzy inference system architecture, [55]; however, this ANFIS model followed a Mamdani ruleset approach to rule-base formulation.

B. Defuzzification and Output Aggregation Combination

As seen earlier, triangle_{O_i} was the triangular output membership function associated with rule i. n_i was the normalized output weight for rule i. r was the total number of rules.

Following those definitions, (5) was redefined as a linear operator to improve computational efficiency. The matrix P was a set of trainable weights representing the triangular output membership functions through a center-of-mass representation, such that $P \in \mathbb{R}^{1\times n}$, $Q \in \mathbb{R}^{n\times r}$, $N \in \mathbb{R}^{r\times 1}$

$$y = \sum_{i=1}^{r} (\text{triangle}_{O_i} \times n_i) \tag{30}$$

$$= \begin{bmatrix} \text{triangle}_{O_1} & \cdots & \text{triangle}_{O_n} \end{bmatrix} \begin{bmatrix} \uparrow & & \uparrow \\ e_1 & \cdots & e_r \\ \downarrow & & \downarrow \end{bmatrix} \begin{bmatrix} n_1 \\ \vdots \\ n_r \end{bmatrix} \tag{31}$$

$$= PQN \tag{32}$$

where e_i was a unit vector representing which output O_n each rule r was associated with, such that $e_{i,j} \in \{0, 1\}$ and $e_i^T e_i = 1$

$$e_i = [0 \quad \cdots \quad 1 \quad \cdots \quad 0]^T. \tag{33}$$

C. Normalization Expansion

The vector W represented the output of each rule obtained through the determination of the degree of matching operation using the product T-norm

$$W = [w_1 \quad \ldots \quad w_r]^T \tag{34}$$

$$N = \begin{bmatrix} n_1 \\ \vdots \\ n_r \end{bmatrix} = \begin{bmatrix} \frac{w_1}{\sum_{i=1}^{r} w_i} \\ \vdots \\ \frac{w_r}{\sum_{i=1}^{r} w_i} \end{bmatrix} = \frac{1}{\sum_{i=1}^{r} w_i} \begin{bmatrix} w_1 \\ \vdots \\ w_r \end{bmatrix} = \frac{1}{\sum_{i=1}^{r} w_i} W \tag{35}$$

$$y = PQN = PQ\left(\frac{1}{\sum_{i=1}^{r} w_i} W\right) = \frac{1}{\sum_{i=1}^{r} w_i} PQW. \tag{36}$$

Given that $w_i \in [0, 1]$, thus, $w_i \geq 0$, $\sum_{i=1}^{r} w_i$ was rewritten as $\|W\|_1 = \sum_{i=1}^{r} |w_i|$. The final step involved the redefinition of the antecedent layer.

D. Antecedent Compression

The product T-norm $(x \times y)$ was used to compute the antecedents. Vector T contained the firing rates for each of the input membership functions $\mu_{k,p}$ over all the k linguistic variables and their p_k number of associated linguistic values. In total, $f = \sum_{i}^{k} p_i$.

The function $A(\cdot) : \mathcal{R}^{f\times 1} \to \mathcal{R}^{r\times 1}$ mapped the f input membership functions to the r rule outputs. As such, $a_i = T_a \times \cdots \times T_q$, represented the specific rule: IF T_a AND T_b AND $\cdots$ AND T_q. In turn, the ANFIS was represented as

$$W = A(T) \tag{37}$$

$$y = \frac{1}{\|A(T)\|_1} PQA(T). \tag{38}$$

Given the mapping function $A(\cdot)$s importance on the rule creation, the size of r indicates the number of rule outputs and thus contributes to the system's computational performance. Techniques, such as the HRBR [39], have been used to reduce

the number of rules r [56], [57], [58] to a minimum by categorizing key aspects of the problem definition and applying expert operator-based weighted categories to classify and organize in a set of hierarchical priority rules.

E. Batch Input

The previous notation for the reduced ANFIS system represented single input, single output notation; however, the system could account for batched inputs. b was the batch index $\bar{x} \in \mathbb{R}^{b\times k}$, $T(\bar{x}) \in \mathbb{R}^{f\times 1}$, $\boldsymbol{\mathcal{T}}(\bar{x}) \in \mathbb{R}^{r\times b}$, $\boldsymbol{\mathcal{N}}(\bar{x}) \in \mathbb{R}^{1\times b}$

$$\boldsymbol{\mathcal{T}}(\bar{x}) = \begin{bmatrix} \uparrow & & \uparrow \\ A(T(\bar{x}_1)) & \cdots & A(T(\bar{x}_b)) \\ \downarrow & & \downarrow \end{bmatrix} \tag{39}$$

$$\boldsymbol{\mathcal{N}}(\bar{x}) = \|\boldsymbol{\mathcal{T}}(\bar{x})\| = [\|\boldsymbol{\mathcal{T}}_1\| \quad \cdots \quad \|\boldsymbol{\mathcal{T}}_b\|]. \tag{40}$$

Hence, the single input ANFIS system in (38) was transformed for the batch inputs case (41)

$$\underbrace{y}_{1\times b} = \left(\underbrace{P}_{1\times n}\underbrace{Q}_{n\times r}\underbrace{\boldsymbol{\mathcal{T}}(\bar{x})}_{r\times b}\right) \oslash \left(\underbrace{\boldsymbol{\mathcal{N}}(\bar{x})}_{1\times b}\right). \tag{41}$$

V. Offline Training

A. Output Weight Optimization Closed Form

This section derives the offline mean square error optimization of the ANFIS. The loss function was defined as

$$l_{\text{sq}}(\hat{y}_i, y_i) = \frac{1}{2}(\hat{y}_i - y_i)^2 \tag{42}$$

$$\hat{R}(P) = \frac{1}{n}\sum_{i=1}^{n} l_{\text{sq}}(\hat{y}_i, y_i) \tag{43}$$

$$= \frac{1}{n}\sum_{i=1}^{n} l_{\text{sq}}\left(\frac{1}{CT_i}PQA(T_i), y_i\right)$$

$$= \frac{1}{2n}\|((PQ\boldsymbol{\mathcal{T}}) \oslash \boldsymbol{\mathcal{N}}) - \mathbf{y}\|_2^2 \tag{44}$$

$$2n\hat{R}(P) = \|((PQ\boldsymbol{\mathcal{T}}) \oslash \boldsymbol{\mathcal{N}}) - \mathbf{y}\|_2^2 \tag{45}$$

$$= \min_{P\in\mathbb{R}} \|((PQ\boldsymbol{\mathcal{T}}) \oslash \boldsymbol{\mathcal{N}}) - \mathbf{y}\|_2^2. \tag{46}$$

The normalization factor was redefined as a broadcasted normalization vector to make the function derivative easier. The broadcast was column-wise to ensure there were n outputs. This broadcast allowed the function model to be rearranged as follows:

$$\boldsymbol{\mathcal{W}} = \underbrace{\begin{bmatrix} \leftarrow & \boldsymbol{\mathcal{N}} & \rightarrow \\ & \vdots & \\ \leftarrow & \boldsymbol{\mathcal{N}} & \rightarrow \end{bmatrix}}_{n\times b}. \tag{47}$$

Together, the gradient of the output gain P and the other variables constant resolved to

$$\begin{aligned} 0 &= \nabla_P \|((PQ\boldsymbol{\mathcal{T}}) \oslash \boldsymbol{\mathcal{N}}) - \boldsymbol{y}\|_2^2 \\ &= \nabla_P \|P((Q\boldsymbol{\mathcal{T}}) \oslash \boldsymbol{\mathcal{W}}) - \boldsymbol{y}\|_2^2 \\ &= 2(P((Q\boldsymbol{\mathcal{T}}) \oslash \boldsymbol{\mathcal{W}}) - \boldsymbol{y})(Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^T \end{aligned} \tag{48}$$

$$\underbrace{\mathbf{y}}_{1\times b}\underbrace{(Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^T}_{b\times n} = \underbrace{P}_{1\times n}\underbrace{(Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})}_{n\times b}\underbrace{(Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^T}_{b\times n}. \tag{49}$$

Consequently, P was found to be

$$\begin{aligned} P &= \boldsymbol{y}(Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^T\left((Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})(Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^T\right)^+ \\ &= \boldsymbol{y}A^T\left(AA^T\right)^+ \\ &= \boldsymbol{y}A^T\left(A^T\right)^+A^+ = \boldsymbol{y}A^+ \end{aligned} \tag{50}$$

$$\underbrace{P}_{1\times n} = \underbrace{\boldsymbol{y}}_{1\times b}\underbrace{(Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^+}_{b\times n} \tag{51}$$

and was represented in a more standard notation of $y = Ax$

$$\underbrace{P^T}_{n\times 1} = \underbrace{\left((Q\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^T\right)^+}_{n\times b}\underbrace{\boldsymbol{y}^T}_{b\times 1}. \tag{52}$$

This result followed a similar approach to the hybrid optimization method used in the initial ANFIS paper [33].

B. Output Weight Symmetrical Constraint

Due to the robotic application this research targeted, the system further employed a parameter reduction for the triangle output weights, namely a symmetric constraint. Most robotic systems assume the system's dynamics are symmetric, so clockwise and counter-clockwise motion pose the same constraints. As a result, the output triangle membership function weights would be symmetric. In turn, Ps gains were symmetrical

$$\begin{aligned} P &= [\text{fast-R} \quad \text{slow-R} \quad \text{zero} \quad \text{slow-L} \quad \text{fast-L}] \\ &= [-\text{fast-L} \quad -\text{slow-L} \quad \text{zero} \quad \text{slow-L} \quad \text{fast-L}]. \end{aligned}$$

The example below demonstrates the symmetric constraint on the angular velocity of a mobile vehicle. Going right (R) implied a negative angular velocity, while going left implied a positive angular velocity, as demonstrated in Fig. 10.

As such, the output gain matrix P was reduced using symmetrical constraints to the parameter set P_r and the symmetry constraint operator M

$$\begin{aligned} P &= [\text{zero} \quad \text{slow-L} \quad \text{fast-L}]\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 \\ -1 & 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \underbrace{P_r}_{1\times q}\underbrace{M}_{q\times n}. \end{aligned} \tag{53}$$

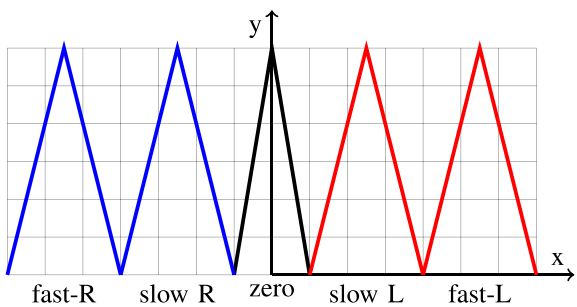

Fig. 10. x_0, x_1 coordinate set demonstration.

The full system with the combination of $\underbrace{M}_{q\times n}\underbrace{Q}_{n\times r} = \underbrace{D}_{q\times r}$ was

$$\underbrace{y}_{1\times b} = \left(\underbrace{P_r}_{1\times q}\underbrace{D}_{q\times r}\underbrace{\boldsymbol{\mathcal{T}}(\bar{x})}_{r\times b}\right) \oslash \left(\underbrace{\boldsymbol{\mathcal{N}}(\bar{x})}_{1\times b}\right). \tag{54}$$

Following this interpretation, the optimal P_r using the least squares regression solution was defined as follows:

$$\underbrace{P_r}_{1\times q} = \underbrace{\boldsymbol{y}}_{1\times b}\underbrace{(D\boldsymbol{\mathcal{T}} \oslash \boldsymbol{\mathcal{W}})^{+}}_{b\times q}. \tag{55}$$

VI. Online Training

A. DDPG Introduction

Reinforcement learning was used to optimize the parameter set of the ANFIS in a dynamic environment. The main reinforcement learning algorithm used to train the ANFIS actor-network was the DDPG model [59].

The standard reinforcement learning setup for discrete action spaces involves an agent acting in discrete time [60], [61]. At each time-step t, the actor receives an observation x_t of its current state in the environment; the actor then decides what action $a_t \in \mathbb{R}^N$ to perform in the environment. The consequences are then defined as a scalar reward r_t.

The actor's behavior is defined by the policy π attributed to it. Given that most environments are stochastic, the policy, $\pi : \mathcal{S} \to \mathcal{P}(\mathcal{A})$, is modeled as a Markov state model with the state space $\mathcal{S}$ and the action space $\mathcal{A} \in \mathbb{R}^N$.

Given the current state s_t and action a_t a reward is attributed to the transition, $r(s_t, a_t)$. In addition, the process transition probabilities are defined by the probability of reaching a specific state $p(s_{t+1}|s_t, a_t)$.

Reinforcement learning algorithms use a recursive approach to compute the Q-value, which approximates the expected value for an action in a specific state. This approach is executed using the Bellman equation. The optimal Q-value can further be simplified using the optimal action $a^*(s)$, which can be found with

$$a^*(s) = \arg\max_a Q^*(s, a). \tag{56}$$

When formulated in a greedy optimization manner, the Bellman equation becomes

$$Q^*(s_t, a_t) = \mathbb{E}\left[r(s_t, a_t) + \gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1})\right]. \tag{57}$$

However, given that Q is often impossible to solve, $Q(s, a)$ is estimated using function approximators, usually a neural network, parameterized by θ^Q, which is optimized using a mean-squared Bellman error function. The set of transitions, $\mathcal{D}$, is defined by $(s_t, a_t, r_t, s_{t+1}, d)$, where d indicates whether the state is an endpoint in the system, such that $d \in \{0, 1\}$. The loss function is, thus, defined as

$$L(\theta^Q, \mathcal{D}) = \mathbb{E}_{s_t, a_t, r_t, s_{t+1}\sim\mathcal{D}}\left[\left(Q(s_t, a_t|\theta^Q) - y_t\right)^2\right] \tag{58}$$

where

$$y_t = r(s_t, a_t) + \gamma(1 - d)Q(s_{t+1}, \mu(s_{t+1})|\theta^Q). \tag{59}$$

Q-learning was previously described on discrete action spaces. This was possible because the a^* optimization was used over a finite action space; however, arg max's equivalent greedy policy in the continuous $a \in \mathcal{R}^n$ would require an optimization process at each step. Optimizing at each timestep becomes prohibitively slow for real-time systems. As such, DDPG makes use of an actor–critic structure. DDPG uses a parameterized actor function $\mu(s|\theta^\mu)$, which represents the system's policy through the deterministic mapping of the states to their equivalent action. As in Q-learning, the critic, which approximates $Q(s, a)$, is optimized through the Bellman equation. As such, the actor is updated using the expected return of its parameters [62]

$$\nabla_{\theta^\mu} J \approx \mathbb{E}\left[\nabla Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s=s_t}\right]. \tag{60}$$

Since the timesteps in the environment are sequentially processed and the samples D are independently and identically distributed (IID), replay buffers are used to solve the IID problem. A replay buffer is a finite cache $\mathcal{R} \subset \mathcal{D}$. While training the actor and critic, mini-batches comprising the replay buffer are sampled uniformly from $\mathcal{R}$, and when the replay buffer is full, the old values are discarded. In addition, DDPG provides a solution against divergent $Q(s, a|\theta^Q)$ given that the updated network also calculates the target value y_t, which causes the Q update to be prone to divergence. As such, target networks are introduced to mitigate this problem. The actor–critic target copy networks are annotated as $\mu'(s|\theta^{\mu'})$ and $Q'(s, a|\theta^{Q'})$, respectively. The target networks then get updated in a time-weighted average fashion to "soft" update the network parameter sets as $\theta'_{t+1} \leftarrow \tau\theta_t + (1 - \tau)\theta'_t$, where $\tau \in [0, 1]$.

B. DDPG Workflow With ANFIS

The DDPG was used as the parameter optimizer for the system, where the Q-value approximator was defined as a deep neural network; however, the parameterized actor function $\mu(s|\theta^\mu)$ was defined as the ANFIS system. As such, the parameter set was defined as $\theta^\mu = \{P, c, w\}$. The use of the ANFIS was comparable to traditional neural network parameterized actors in its universal function approximation ability; however, through

TABLE I
CLEARPATH JACKAL ACTION SPACE

Action	Symbol	Min	Max
Angular velocity	ω	−4.0	4.0

the ANFIS, there was a drastic parameter reduction and control over the system's characteristics.

The control over the system's characteristics stems from the ability to modify the membership functions. As such, ANFIS is advantageous for understanding the input–output relationship compared to large neural networks whose weights can be difficult to effectually alter without prior knowledge of the consequences [12]. In addition, the ruleset is defined in an intuitive and constrained manner, which provides an easier understanding of each rule set's impact and its contributions to the outputs. This makes it easier to manipulate and troubleshoot the system by using a masking operation to understand the output of each ruleset. The ANFIS using fuzzy logic can be used as a universal approximation [63], similar to neural networks [64], [65], [66], [67], [68]. Thus, a minimum ANFIS definition can be derived such that one can approximate the state-to-action policy space.

VII. ONLINE RESULTS

A. Environment

The ANFIS was evaluated in simulation as a motion controller to a skid-steer motion model unmanned ground vehicle (UGV). Table I defines the system's inputs. The linear velocity was set to a constant value of 1.0 $\frac{m}{s}$.

The Clearpath Jackal was the UGV used. The ANFIS acted based on the current state of the robot along the path. The state of the robot was defined through five error functions.

The waypoints were defined as a specific location in the world. The path p represented the sequential set of waypoints. To follow the robot's progress, particular waypoints were referenced: x_p was the waypoint the robot had recently passed, x_c was the waypoint the robot was heading toward, and x_f, represented the next/future waypoint that the robot would reach. Finally, x described the robot's location in the world.

The five error states of the robot were represented as follows.

1) *Distance target:* The distance to reach the next waypoint

$$d_t = \|x_c - x\|_2 \in [0, \infty). \tag{61}$$

2) *Distance line:* The projected remaining distance of x on the waypoint trajectory $x_p \to x_c$

$$a = x_c - x_p \tag{62}$$

$$b = x - x_p \tag{63}$$

$$d_l = \|b - \left(\frac{a \cdot b}{a \cdot a}\right) a\|_2 \in (-\infty, \infty). \tag{64}$$

3) *Theta lookahead:* The angle difference between the next path segment's heading and the current heading

$$\theta_r = \arctan 2(x_y, x_x) \tag{65}$$

$$\theta_f = \arctan 2(x_{f,y} - x_{c,y}, x_{f,x} - x_{c,x}) \tag{66}$$

$$\theta_{\text{le}} = \theta_f - \theta_r \in (-\pi, \pi). \tag{67}$$

4) *Theta near:* The angle difference between the heading of the current path segment and the agent's current heading

$$\theta_n = \arctan 2(x_{f,y} - x_{c,y}, x_{f,x} - x_{c,x}) \tag{68}$$

$$\theta_{\text{ne}} = \theta_n - \theta_r \in (-\pi, \pi). \tag{69}$$

5) *Theta far:* The angle difference between the projected recovery point and the agent's heading, with $k \in [0, 1]$

$$x_{\text{proj}} = \frac{a \cdot b}{a \cdot a} a + x_p \tag{70}$$

$$x_{\text{ft}} = k x_{\text{proj}} + (1 - k) x_c \tag{71}$$

$$\theta_{\text{ft}} = \arctan 2(x_{\text{ft},y} - x_y, x_{ft,x} - x_x) \tag{72}$$

$$\theta_{\text{fe}} = \theta_{\text{ft}} - \theta_r \in (-\pi, \pi). \tag{73}$$

The reward for the system was defined as

$$\text{dist}_r = \frac{1}{|d_t| + a} - \frac{1}{a} \in \left(-\frac{1}{a}, 0\right] \tag{74}$$

$$\theta_{\text{ne},r} = \frac{1}{|\theta_{\text{ne}}| + b} - \frac{1}{b} \in \left(-\frac{1}{b}, 0\right] \tag{75}$$

$$r = \text{dist}_r + \theta_{\text{ne},r}. \tag{76}$$

B. Model Parameters

The rule-base was defined in the mapping function A with the rule-to-output relationship defined in matrix Q. To save space, "left" and "right" were reduced to "l" and "r," respectively. The ordered input membership values were represented in vector T

$$\begin{aligned}
T_{d_t} &= \begin{bmatrix}\text{near} & \text{far}\end{bmatrix} \\
T_{d_l} &= \begin{bmatrix}\text{far}_l & \text{near}_l & \text{close}_l & \text{zero} & \text{close}_r & \text{near}_r & \text{far}_r\end{bmatrix} \\
T_{\boldsymbol{\theta}_{\text{le}}} &= \begin{bmatrix}\text{far}_l & \text{close}_l & \text{zero} & \text{close}_r & \text{far}_r\end{bmatrix} \\
T_{\boldsymbol{\theta}_{\text{fe}}} &= \begin{bmatrix}\text{far}_l & \text{close}_l & \text{zero} & \text{close}_r & \text{far}_r\end{bmatrix} \\
T_{\boldsymbol{\theta}_{\text{ne}}} &= \begin{bmatrix}\text{far}_l & \text{near}_l & \text{zero} & \text{near}_r & \text{far}_r\end{bmatrix} \\
T &= \begin{bmatrix}T_{d_t} & | & T_{d_l} & | & T_{\boldsymbol{\theta}_{\text{le}}} & | & T_{\boldsymbol{\theta}_{\text{fe}}} & | & T_{\boldsymbol{\theta}_{\text{ne}}}\end{bmatrix}^T.
\end{aligned} \tag{77}$$

The output matrix weight P

$$\text{Pr}_{\dot{\boldsymbol{\theta}}_{\boldsymbol{R}}} = \begin{bmatrix}\text{zero} & \text{left}_1 & \text{left}_2 & \text{left}_3 & \text{left}_4\end{bmatrix}. \tag{78}$$

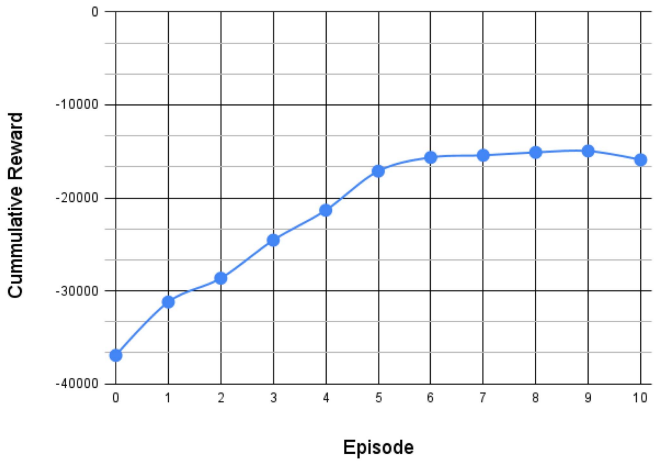

Fig. 11. Reward over episodes.

The output query matrix Q

$$Q^T = \begin{bmatrix}
\mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} \\
\mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} \\
\mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} \\
\mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\
\mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 \\
0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 \\
0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} \\
0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1}
\end{bmatrix}. \tag{79}$$

With the additional output symmetry matrix M

$$M = \begin{bmatrix}
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & -1 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & -1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
-1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{bmatrix}. \tag{80}$$

The input rule-base mapping function $A(\cdot)$

$$A = \begin{bmatrix}
d_{t,\text{Near}} & \times & \theta_{\text{le,Far-left}} & & \\
d_{t,\text{Near}} & \times & \theta_{\text{le,Near-left}} & & \\
d_{t,\text{Near}} & \times & \theta_{\text{le,Zero}} & & \\
d_{t,\text{Near}} & \times & \theta_{\text{le,Near-right}} & & \\
d_{t,\text{Near}} & \times & \theta_{\text{le,Far-right}} & & \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-left}} & \times & \theta_{\text{ne,Far-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-left}} & \times & \theta_{\text{ne,Near-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-left}} & \times & \theta_{\text{ne,Zero}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-left}} & \times & \theta_{\text{ne,Near-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-left}} & \times & \theta_{\text{ne,Far-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-right}} & \times & \theta_{\text{ne,Far-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-right}} & \times & \theta_{\text{ne,Near-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-right}} & \times & \theta_{\text{ne,Zero}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-right}} & \times & \theta_{\text{ne,Near-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Far-right}} & \times & \theta_{\text{ne,Far-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-left}} & \times & \theta_{\text{fe,Far-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-left}} & \times & \theta_{\text{fe,Near-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-left}} & \times & \theta_{\text{fe,Zero}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-left}} & \times & \theta_{\text{fe,Near-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-left}} & \times & \theta_{\text{fe,Far-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-left}} & \times & \theta_{\text{fe,Far-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-left}} & \times & \theta_{\text{fe,Near-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-left}} & \times & \theta_{\text{fe,Zero}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-left}} & \times & \theta_{\text{fe,Near-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-left}} & \times & \theta_{\text{fe,Far-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Zero}} & \times & \theta_{\text{fe,Far-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Zero}} & \times & \theta_{\text{fe,Near-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Zero}} & \times & \theta_{\text{fe,Zero}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Zero}} & \times & \theta_{\text{fe,Near-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Zero}} & \times & \theta_{\text{fe,Far-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-right}} & \times & \theta_{\text{fe,Far-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-right}} & \times & \theta_{\text{fe,Near-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-right}} & \times & \theta_{\text{fe,Zero}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-right}} & \times & \theta_{\text{fe,Near-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Close-right}} & \times & \theta_{\text{fe,Far-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-right}} & \times & \theta_{\text{fe,Far-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-right}} & \times & \theta_{\text{fe,Near-left}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-right}} & \times & \theta_{\text{fe,Zero}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-right}} & \times & \theta_{\text{fe,Near-right}} \\
d_{t,\text{Far}} & \times & d_{l,\text{Near-right}} & \times & \theta_{\text{fe,Far-right}}
\end{bmatrix} \tag{81}$$

The system was then trained using a DDPG with the Adam optimizer with learning rates 0.001 and 0.0001 for the critic and actor parameters, respectively. The critic was represented as a deep neural network and the actor was the ANFIS. The symmetry axis c was uniformly set to 0 for all connected

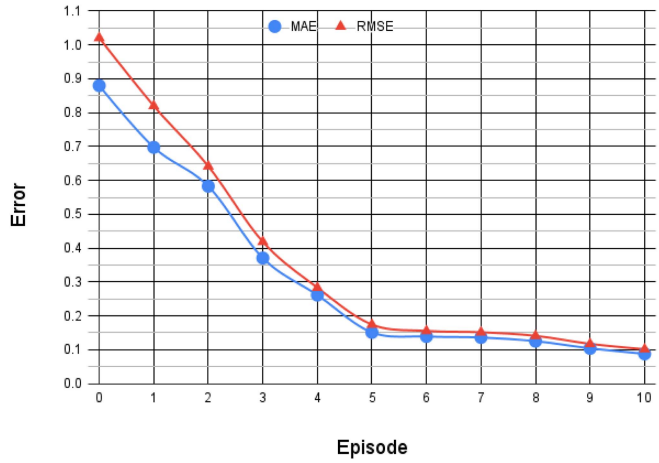

Fig. 12. MAE and RMSE over episodes.

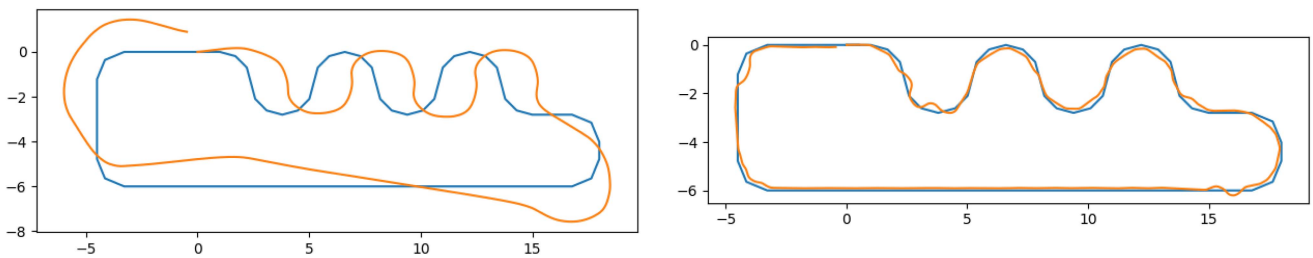

Fig. 13. Start and end ANFIS model.

TABLE II
INITIAL CONNECTED MEMBERSHIP STATE

Variable	Type	Weights
d_l	S $[0, \infty)$	$0, 0.701, 0, 0.701, 0, 0.701$
d_t	U $(-\infty, \infty)$	$0, 1$
θ_{le}	B $[-\pi, \pi]$	$0, 0.701, 0, 0.701, 1.463$
θ_{fe}	B $[-\pi, \pi]$	$0, 0.701, 0, 0.701, 1.463$
θ_{ne}	B $[-\pi, \pi]$	$0, 0.701, 0, 0.701, 1.463$
P		$0, 0.25, 0.5, 0.75, 1$

TABLE III
FINAL CONNECTED MEMBERSHIP STATE

Variable	Weights
d_l	$0, 0.2089, 0, 0.1965, 0, 0.1989$
d_t	$0.0826, 0.9491$
θ_{le}	$0, 0.6388, 0, 0.6661, 1.522$
θ_{fe}	$0, 0.4631, 0, 0.4676, 1.709$
θ_{ne}	$0.0194, 0.3396, -0.0311, 0.5272, 1.832$
P	$0, 0.05334, 0.709, 1.036, 1.655$

membership functions. Each linguistic variable's respective parameters and membership type were initially assigned to the values displayed in Table II. Three membership types were utilized: unrestricted (U), single-sided (S), and bounded (B), each representing membership functions with domains $(-\infty, \infty)$, $[c, \infty)$, and $[a, b]$, respectively, as described in Sections III-B3, III-B4, and III-B5.

The performance of the proposed system was evaluated by analyzing the training process results. The cumulative reward function of the system reached an equilibrium point within a few episodes, typically between 6 and 10 episodes, as depicted in Fig. 11. Consistent improvements were documented in the graphs of the mean absolute error (MAE) and root mean squared error (RMSE) throughout the episodes. The MSE and RMSE were calculated relative to the distance line d_l input parameter, which indicated the robot's proximity to the trajectory. The results obtained from the experiments are presented in Fig. 12. The initial system parameters enabled the robot to achieve an MAE and RMSE of 0.88 m and 1.02 m, respectively. However, after the training process, the system achieved significantly improved results, with an MAE and RMSE of 0.087 m and 0.10 m, respectively. This represented a ten-fold improvement in the system's trajectory following capability, as demonstrated by the trained ANFIS.

The accuracy of the ANFIS model after tuning was assessed by examining how well it followed the trajectory. As shown in Fig. 13, the new ANFIS model was observed to accurately track the course, without any delays or inaccuracies, even during the initial curved section. Overall, the results indicate that the proposed system effectively enabled the robot to track the trajectory accurately. Table III displays the final parameter set values.

VIII. Conclusion

In conclusion, the results of this research demonstrated the effectiveness of using the ANFIS as a controller in applications with symmetrically constrained problems. The ANFIS model required fewer parameters and less computation than typical neural network approaches, leading to faster parameter convergence and stable system characteristics. The human-interpretable and malleable nature of the ANFIS also made it a desirable option for motion controllers. The ANFIS's rule-set and nonlinear membership functions made mitigating the closed box approach of neural network-based controllers possible through its membership function explainability.

For the Clearpath Jackal robot example, the experimental results showed that the ANFIS model converged quickly, typically within 6 to 10 episodes of training, from an initial MAE of 0.88 m and RMSE of 1.02 m to a final MAE of 0.087 m and RMSE of 0.10 m, indicating a significant improvement in the ANFIS's trajectory following performance. The waypoint trajectory generator will be refined to follow a continuous and differentiable trajectory, leading to smoother motion between waypoints.

Future work in this area will involve implementing systematic optimization in constructing and minimizing the rule-base, allowing for a reduction in manual work required to derive the rule-base of the underlying system. The approach could depend on developing stability criteria enabling the system to understand the parameter set domain needed to maintain stability. Adaptive ways to define the rule-base through a rule-base encoder and decoder-based neural network or some other Lyapunov-based stability criteria will be explored. Finally, the use of joint sigmoid difference membership functions instead of trapezoidal membership functions will be derived to have a smoother differentiable system.

Overall, the results of this study suggest that the ANFIS has the potential to be an effective controller, with promising avenues for future research and development.

References

[1] J. A. D. Bagnell and J. Schneider, "Autonomous helicopter control using reinforcement learning policy search methods," in *Proc. Int. Conf. Robot. Autom.*, May 2001, pp. 1615–1620.

[2] M. Deisenroth, C. Rasmussen, and D. Fox, "Learning to control a low-cost manipulator using data-efficient reinforcement learning," in *Robot: Sci. and Syst. VII*. San Francisco, CA, USA: Robotics: Science and Systems Foundation, Jun. 2011, pp. 57–64. [Online]. Available: http://www.roboticsproceedings.org/rss07/p08.pdf

[3] V. Gullapalli, J. A. Franklin, and H. Benbrahim, "Acquiring robot skills via reinforcement learning," *IEEE Control Syst.*, vol. 14, no. 1, pp. 13–24, Feb. 1994. [Online]. Available: https://ieeexplore.ieee.org/document/257890/

[4] Y. He, P. Veličković, P. Lio, and A. Deac, "Continuous neural algorithmic planners," in *Proc. 1st Learn. Graphs Conf.*, 2022, pp. 1–13.[Online]. Available: https://openreview.net/forum?id=60avttW0Mv

[5] D. Jarrett, C. Tallec, F. Altché, T. Mesnard, R. Munos, and M. Valko, "Curiosity in hindsight," Nov. 2022. [Online]. Available: http://arxiv.org/abs/2211.10515

[6] N. Kohl and P. Stone, "Policy gradient reinforcement learning for fast quadrupedal locomotion," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2004, pp. 2619–2624.

[7] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Sci. Robot.*, vol. 7, no. 62, Jan. 2022, Art. no. eabk2822. [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.abk2822

[8] H. Miyamoto et al., "A Kendama learning robot based on bi-directional theory," *Neural Netw.*, vol. 9, no. 8, pp. 1281–1302, 1996.

[9] I. OpenAI et al., "Solving Rubik's cube with a robot hand," 2019. [Online]. Available: https://arxiv.org/abs/1910.07113

[10] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Sci. Robot.*, vol. 5, no. 47, Oct. 2020. [Online]. Available: http://dx.doi.org/10.1126/scirobotics.abc5986

[11] D. Steinkraus, I. Buck, and P. Simard, "Using GPUs for machine learning algorithms," in *Proc. 8th Int. Conf. Document Anal. Recognit.*, 2005, pp. 1115–1120. [Online]. Available: https://ieeexplore.ieee.org/document/1575717/

[12] P. Gohel, P. Singh, and M. Mohanty, "Explainable AI: Current status and future directions," Jul. 2021. [Online]. Available: http://arxiv.org/abs/2107.07045

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, Jun. 2015, pp. 779–788. [Online]. Available: https://arxiv.org/abs/1506.02640v5

[14] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel, "Deep spatial autoencoders for visuomotor learning," in *Proc. IEEE Int. Conf Robot. Autom.*, Sep. 2015, pp. 512–519, [Online]. Available: https://arxiv.org/abs/1509.06113v3

[15] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine, "How to train your robot with deep reinforcement learning; lessons we've Learned," *Int. J. Robot. Res.*, vol. 40, no. 4/5, pp. 698–721, Feb. 2021, doi: 10.1177/0278364920987859.

[16] L. Pinto and A. Gupta, "Supersizing self-supervision: Learning to grasp from 50 K tries and 700 robot hours," in *Proc. IEEE Int. Conf. Robot. Autom.*, Sep. 2015, pp. 3406–3413. [Online]. Available: https://arxiv.org/abs/1509.06825v1

[17] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *J. Mach. Learn. Res.*, vol. 17, pp. 1–40, Apr. 2015. [Online]. Available: https://arxiv.org/abs/1504.00702v5

[18] Z. Li, Y. Mu, Z. Sun, S. Song, J. Su, and J. Zhang, "Intention understanding in human–robot interaction based on visual-NLP Semantics," *Front. Neurorobot.*, vol. 14, Feb. 2021, Art. no. 121.

[19] C. Matuszek, "Grounded language learning: Where robotics and NLP meet," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 5687–5691. [Online]. Available: https://www.ijcai.org/proceedings/2018/810

[20] J. Y. Chai, P. Hong, and M. X. Zhou, "A probabilistic approach to reference resolution in multimodal user interfaces," in *Proc. 9th Int. Conf. Intell. User Interfaces*, Jan. 2004, pp. 70–77, doi: 10.1145/964442.964457.

[21] S. Tellex et al., "Understanding natural language commands for robotic navigation and mobile manipulation," in *Proc. AAAI Conf. Artif. Intell.*, Aug. 2011, pp. 1507–1514. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/7979

[22] S. Amiri, S. Wei, S. Zhang, J. Sinapov, J. Thomason, and P. Stone, "Multi-modal predicate identification using dynamically learned robot controllers," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 4638–4645.

[23] T. Kollar, V. Perera, D. Nardi, and M. M. Veloso, "Learning environmental knowledge from task-based human-robot dialog," 1984. [Online]. Available: https://kilthub.cmu.edu/articles/journal_contribution/Learning_Environmental_Knowledge_from_Task-Based_Human-Robot_Dialog/6606857

[24] D. Shah, B. Osinski, B. Ichter, and S. Levine, "LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action," Jul. 2022. [Online]. Available: https://arxiv.org/abs/2207.04429v2

[25] A. Boteanu, T. Howard, J. Arkin, and H. Kress-Gazit, "A model for verifiable grounding and execution of complex natural language instructions," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2016, pp. 2649–2654. [Online]. Available: https://ieeexplore.ieee.org/document/7759412/

[26] J. Fasola and M. J. Mataric, "Using socially assistive human–robot interaction to motivate physical exercise for older adults," *Proc. IEEE*, vol. 100, no. 8, pp. 2512–2526, Aug. 2012. [Online]. Available: https://ieeexplore.ieee.org/document/6235980/

[27] W. Huang et al., "Inner monologue: Embodied reasoning through planning with language models," Jul. 2022. [Online]. Available: https://inner-monologue.github.io/ http://arxiv.org/abs/2207.05608

[28] J. N. Pires, *Robot Manipulators and Control Systems*. Boston, MA, USA: Springer, 2007, pp. 35–107, doi: 10.1007/978-0-387-23326-0_2.

[29] S. Sastry, *Lyapunov Stability Theory*. New York, NY, USA: Springer, 1999, pp. 182–234, doi: 10.1007/978-1-4757-3108-8_5.

[30] M. A. Shoorehdeli, M. Teshnehlab, and A. Sedigh, "Stable learning algorithm approaches for ANFIS as an identifier," *IFAC Proc. Volumes*, vol. 41, no. 2, pp. 7046–7051, Jan. 2008. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1474667016400789

[31] G. Ma and W. Cai, "Design and application of model free controller," in *Proc. Int. Conf. Electric Inf. Control Eng.*, Apr. 2011, pp. 2058–2061. [Online]. Available: https://ieeexplore.ieee.org/document/5778310/

[32] M. Fliess and C. Join, "Model-free control," *Int. J. Control*, vol. 86, no. 12, pp. 2228–2252, May 2013, doi: 10.1080/00207179.2013.810345.

[33] J.-S. Jang, "ANFIS: Adaptive-network-based fuzzy inference system," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, no. 3, pp. 665–685, May/Jun. 1993. [Online]. Available: https://ieeexplore.ieee.org/document/256541/

[34] A. Pandey, "Multiple mobile robots navigation and obstacle avoidance using minimum rule based ANFIS network controller in the cluttered environment," *Int. J. Adv. Robot. Autom.*, vol. 1, no. 1, pp. 1–11, Feb. 2016. [Online]. Available: http://www.symbiosisonlinepublishing.com/robotics-automation/robotics-automation02.php

[35] R. Velazquez and A. Lay-Ekuakille, "A review of models and structures for wheeled mobile robots: Four case studies," 2011, pp. 524–529. [Online]. Available: https://ieeexplore.ieee.org/document/6088568/

[36] G. Campion and W. Chung, *Wheeled Robots*. Berlin, Germany: Springer, 2008, pp. 391–410. [Online]. Available: http://link.springer.com/10.1007/978-3-540-30301-5_18

[37] L. A. Zadeh, "Outline of a new approach to the analysis of complex systems and decision processes," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC- 3, no. 1, pp. 28–44, Jan. 1973. [Online]. Available: https://ieeexplore.ieee.org/document/5408575/

[38] L. Zadeh, "Fuzzy sets," *Inf. Control*, vol. 8, no. 3, pp. 338–353, Jun. 1965. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S001999586590241X

[39] W. R. Norris, "Hierarchical rule-base reduction for a fuzzy logic based human operator performance model," *Intell. Eng. Syst. Through Artif. Neural Netw.*, vol. 11, pp. 285–294, 2001.

[40] W. Norris, "A design framework for qualitative human-in-the-loop system development," Ph.D. dissertation, Univ. illinois Urbana-Champaign, Ind. Enterprise Syst. Eng., Champaign, IL, USA, 2001.

[41] W. Norris, Q. Zhang, and R. Sreenivas, "Rule-base reduction for a fuzzy human operator performance model," *Appl. Eng. Agriculture*, vol. 22, no. 4, pp. 611–618, Jul. 2006.

[42] J. Xu and W. R. Norris, "Hierarchical fuzzy control design and stability analysis for steering-control based vehicles," in *Proc. 4th Int. Conf. Autom. Control Robot. Eng.*, 2019, Art. no. 40, doi: 10.1145/3351917.3351961.

[43] G. V. S. Raju, J. Zhou, and R. A. Kisner, "Hierarchical fuzzy control," *Int. J. Control*, vol. 54, pp. 1201–1216, 1991. [Online]. Available: http://www.tandfonline.com/doi/abs/10.1080/00207179108934205

[44] M. Brown, K. Bossley, D. Mills, and C. Harris, "High dimensional neurofuzzy systems: Overcoming the curse of dimensionality," vol. 4, pp. 2139–2146, 1995. [Online]. Available: https://ieeexplore.ieee.org/document/409976/

[45] M.-L. Lee, H.-Y. Chung, and F.-M. Yu, "Modeling of hierarchical fuzzy systems," *Fuzzy Sets Syst.*, vol. 138, pp. 343–361, 2003. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0165011402005171

[46] V. Torra, "A review of the construction of hierarchical fuzzy systems," *Int. J. Intell. Syst.*, vol. 17, pp. 531–543, 5 2002. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1002/int.10036

[47] L. T. Kóczy and L. Muresan, *Interpolation in Hierarchical Rule-Bases with Normal Conclusions*. Berlin, Germany: Springer, 2002, vol. 2275, pp. 34–39. [Online]. Available: http://link.springer.com/10.1007/3-540-45631-7_5

[48] J. Leventides and A. Bounas, "An approach to the selection of fuzzy connectives in terms of their additive generators," *Fuzzy Sets Syst.*, vol. 126, pp. 219–224, 2002. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0165011401000872

[49] E. P. Klement, R. Mesiar, and E. Pap, "Triangular norms: Basic notions and properties," in *Logical Algebraic Analytic Probabilistic Aspects Triangular Norms*. Amsterdam, The Netherlands: Elsevier, 2005pp. 17–60.

[50] D. Gómez, J. T. Rodríguez, J. Montero, H. Bustince, and E. Barrenechea, "n-dimensional overlap functions," *Fuzzy Sets Syst.*, vol. 287, pp. 57–75, 2016. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0165011414005417

[51] G. Marra, F. Giannini, M. Diligenti, M. Maggini, and M. Gori, "T-norms driven loss functions for machine learning," Jul. 2019. [Online]. Available: http://arxiv.org/abs/1907.11468

[52] S. H. Khairuddin, M. H. Hasan, and M. A. Hashmani, "Integration of cluster centers and Gaussian distributions in fuzzy C-means for the construction of trapezoidal membership function," *Math. Statist.*, vol. 8, no. 5, pp. 559–565, 2020.

[53] E. H. Ruspini, "A new approach to clustering," *Inf. Control*, vol. 15, pp. 22–32, 1969. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0019995869905919

[54] P. Codara, O. M. D'Antona, and V. Marra, "An analysis of Ruspini partitions in Gödel logic," *Int. J. Approx. Reasoning*, vol. 50, pp. 825–836, 2009. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0888613X09000395

[55] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE Trans. Syst., Man Cybernet.*, vol. SMC-15, no. 1, pp. 116–132, Jan./Feb. 1985.

[56] L. Mao, Q. Chen, and J. Sun, "Construction and optimization of fuzzy rule-based classifier with a swarm intelligent algorithm," *Math. Problems Eng.*, vol. 2020, 2020, Art. no. 9319364, doi: 10.1155/2020/9319364.

[57] S. V. Wong and A. M. S. Hamouda, "Optimization of fuzzy rules design using genetic algorithm," *Adv. Eng. Softw.*, vol. 31, no. 4, pp. 251–262, 2000. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S096599789900054X

[58] M. Fathi-Torbaghan and L. Hildebrand, "Model-free optimization of fuzzy rule-based systems using evolution strategies," *IEEE Trans. Syst., Man, Cybern. B., Cybern.*, vol. 27, no. 2, pp. 270–277, Apr. 1997.

[59] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in *Proc. 4th Int. Conf. Learn. Representations*, Sep. 2015, pp. 1–14. [Online]. Available: https://arxiv.org/abs/1509.02971v6

[60] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–33, Feb. 2015. [Online]. Available: https://www.nature.com/articles/nature14236

[61] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. 31st Int. Conf. Mach. Learn.*, Bejing, China, 2014, pp. 387–395. [Online]. Available: https://proceedings.mlr.press/v32/silver14.html

[62] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, no. 3/4, pp. 229–256, May 1992. [Online]. Available: https://link.springer.com/article/10.1007/BF00992696

[63] J. Castro, "Fuzzy logic controllers are universal approximators," *IEEE Trans. Syst., Man, Cybern.*, vol. 25, no. 4, pp. 629–635, Apr. 1995. [Online]. Available: https://ieeexplore.ieee.org/document/370193/

[64] R. Brüel Gabrielsson, "Universal function approximation on graphs," in *Adv. Neural Inf. Process. Syst.*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds. Red Hook, NY, USA: Curran Associates, Inc., 2020, pp. 19762–19772. [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/e4acb4c86de9d2d9a41364f93951028d-Paper.pdf

[65] A. M. Schäfer and H. G. Zimmermann, "Recurrent neural networks are universal approximators," in *Lecture Notes in Computer Science (Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Berlin, Germany: Springer, 2006, pp. 632–640. [Online]. Available: https://link.springer.com/chapter/10.1007/11840817_66

[66] A. Heinecke, J. Ho, and W.-L. Hwang, "Refinement and universal approximation via sparsely connected ReLU convolution nets," *IEEE Signal Process. Lett.*, vol. 27, pp. 1175–1179, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9126188/

[67] D.-X. Zhou, "Universality of deep convolutional neural networks," *Appl. Comput. Harmon. Anal.*, vol. 48, no. 2, pp. 787–794, May 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1063520318302045

[68] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Math. Control Signals, Syst.*, vol. 2, no. 4, pp. 303–314, Dec. 1989. [Online]. Available: https://link.springer.com/article/10.1007/BF02551274